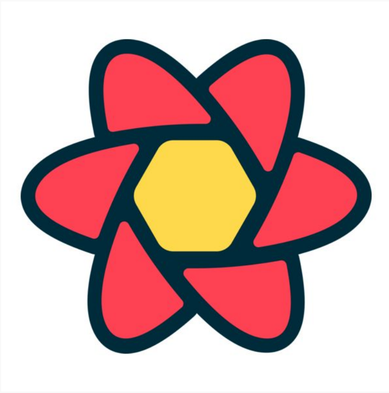
Google Developer Group
Universitas Sriwijaya

Learning 13

Data Fetching with TanStack Query

Efficient data fetching and server state management



Database (Supabase / PostgreSQL)

Supabase stores application data in a PostgreSQL database.

```sql
CREATE TABLE students (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  major TEXT,
  year INT
);
```

```sql
INSERT INTO students (name, major, year) VALUES
('Andi Saputra', 'Informatics', 2022),
('Budi Santoso', 'Information Systems', 2021),
('Citra Lestari', 'Computer Science', 2023);
```

This table will be accessed by our React application.

Fetching Data from Supabase

Supabase provides a JavaScript client to access database data.

Example Query:

```
const { data, error } = await supabase
  .from("students")
  .select("*")
```

Example returned data:

```
[
  { id: 1, name: "Andi Saputra", major: "Informatics" },
  { id: 2, name: "Budi Santoso", major: "Information Systems" }
]
```

However, managing fetching manually in React can become complex.

Fetching Data with useEffect (Traditional Way)

Without additional libraries, data fetching often looks like this:

Client State:

- UI state
- form input
- modal open/close

Server State:

- database data
- remote API responses
- shared backend resources

Server state requires:

- fetching
- caching
- synchronization

Client State vs Server State

Not all state is the same

```
useEffect(() => {
  async function fetchStudents() {
    const { data } = await supabase
      .from("students")
      .select("*")

    setStudents(data)
  }

  fetchStudents()
}, [])
```

Problems:

- repetitive boilerplate
- manual loading state
- manual error handling
- no caching

What is TanStack Query?

TanStack Query is a server state management library for React.

It simplifies:

- data fetching
- caching
- background refetching
- loading & error states

Many modern React applications use it for API data management.

Installing TanStack Query

Install the library:

```
npm install @tanstack/react-query
```

Initialize the Query Client.

```
import { QueryClient } from "@tanstack/react-query"

const queryClient = new QueryClient()
```

Wrap the application with *QueryClientProvider*

QueryClientProvider Setup

Supabase stores application data in a PostgreSQL database.

```
<QueryClientProvider client={queryClient}>
  <App />
</QueryClientProvider>
```

This enables TanStack Query features across the application.

All components can now use queries.

Fetching Data with useQuery

TanStack Query provides the useQuery hook

```
const { data, isLoading, error } = useQuery({
  queryKey: ["students"],
  queryFn: fetchStudents
})
```

Features automatically handled:

- loading state
- error state
- caching

Fetching from Supabase with TanStack Query

Example fetch function:

```
async function fetchStudents() {
  const { data, error } = await supabase
    .from("students")
    .select("*")

  if (error) throw error

  return data
}
```

Use inside a query:

```
const { data } = useQuery({
  queryKey: ["students"],
  queryFn: fetchStudents
})
```

Key Benefits of TanStack Query

TanStack Query provides:

- automatic caching
- background refetching
- built-in loading & error state
- simplified server state management

Challenge! (FE Member Only*)

1. Buat sebuah simple web menggunakan React dengan tema bebas
2. Setup project Supabase dan buat minimal satu tabel di database
3. Fetch dan tampilkan data dari Supabase menggunakan TanStack Query
4. Implementasikan apa yang telah dipelajari (Autentikasi bersifat opsional)
5. Lakukan pull request ke Frontend Development GDGoC UNSRI 25/26 untuk memasukkan link repo github kamu di folder /13-supabase-fetching/task.txt

Selamat Mencoba ~

Google Developer Group
Universitas Sriwijaya

Thank You !